

✓ Mastering TensorFlow: Day 6 - Evaluating Model Performance

✓ Recap of Day 5

Yesterday, we built a simple neural network and trained it on a small dataset. Today, we'll focus on evaluating the performance of that model.

```
import tensorflow as tf
import numpy as np
```

```
# Creating a simple dataset with NumPy
data = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
labels = np.array([0, 1, 2])

# Converting the data to TensorFlow tensors
data_tensor = tf.constant(data, dtype=tf.float32)
labels_tensor = tf.constant(labels, dtype=tf.int32)

# Normalizing the data
normalized_data_tensor = data_tensor / 9.0
```

```
# Rebuilding the simple neural network model
model = tf.keras.Sequential([
    tf.keras.layers.Dense(units=3, input_shape=[3]), # Input layer
    tf.keras.layers.Dense(units=3, activation='relu'), # Hidden layer
    tf.keras.layers.Dense(units=1) # Output layer
])

# Compiling the model
model.compile(optimizer='adam', loss='mean_squared_error')

# Training the model again
model.fit(normalized_data_tensor, labels_tensor, epochs=10)
```



Epoch 1/10

```
/usr/local/lib/python3.10/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

1/1 ————— 4s 4s/step - loss: 1.6667

Epoch 2/10

1/1 ————— 0s 105ms/step - loss: 1.6647

Epoch 3/10

1/1 ————— 0s 111ms/step - loss: 1.6627

Epoch 4/10

1/1 ————— 0s 71ms/step - loss: 1.6607

Epoch 5/10

1/1 ————— 0s 146ms/step - loss: 1.6587

Epoch 6/10

```
Epoch 6/10
1/1 ----- 0s 113ms/step - loss: 1.6567
Epoch 7/10
1/1 ----- 0s 128ms/step - loss: 1.6547
Epoch 8/10
1/1 ----- 0s 106ms/step - loss: 1.6527
Epoch 9/10
1/1 ----- 0s 89ms/step - loss: 1.6507
Epoch 10/10
1/1 ----- 0s 41ms/step - loss: 1.6488
<keras.src.callbacks.history.History at 0x7c5a0f1dac20>
```

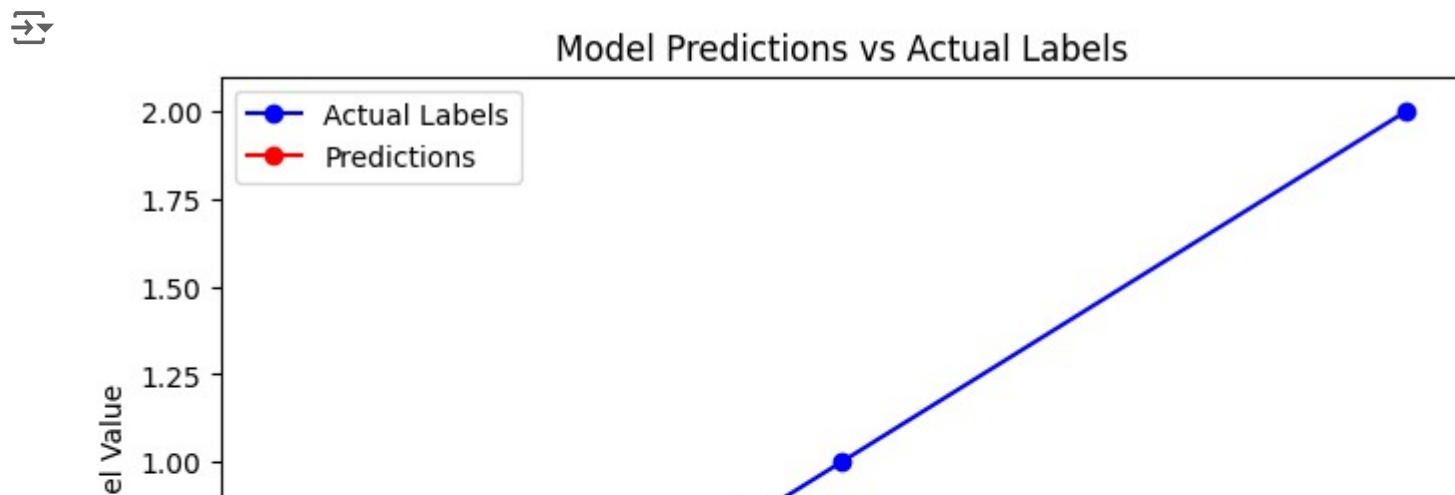
```
# Evaluating the model
loss = model.evaluate(normalized_data_tensor, labels_tensor)
print("Model Loss:", loss)

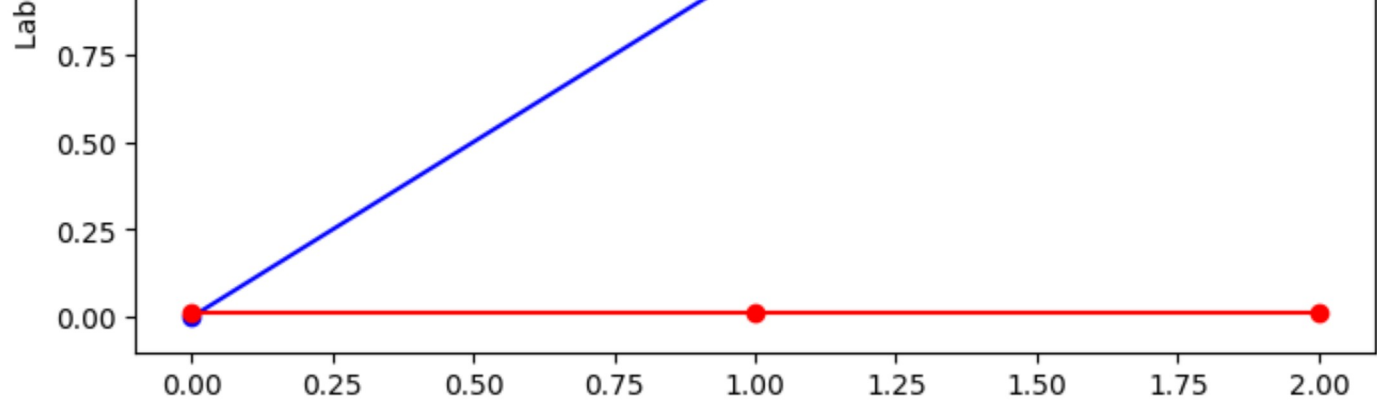
# Making predictions to see how well the model performs
predictions = model.predict(normalized_data_tensor)
print("Predictions:\n", predictions)
print("Actual Labels:\n", labels_tensor.numpy())
```

```
➞ 1/1 ----- 0s 105ms/step - loss: 1.6468
Model Loss: 1.646773338317871
1/1 ----- 0s 49ms/step
Predictions:
[[0.00999667]
 [0.00999667]
 [0.00999667]]
Actual Labels:
[0 1 2]
```

```
import matplotlib.pyplot as plt

# Plotting predictions vs actual labels
plt.figure(figsize=(8, 5))
plt.plot(labels_tensor.numpy(), 'bo-', label="Actual Labels")
plt.plot(predictions, 'ro-', label="Predictions")
plt.title("Model Predictions vs Actual Labels")
plt.xlabel("Sample Index")
plt.ylabel("Label Value")
plt.legend()
plt.show()
```





✓ Conclusion

Today, we evaluated our simple neural network's performance by examining its loss and comparing its predictions to actual labels. Tomorrow, we'll explore techniques to improve model performance and reduce loss. Stay tuned! 🚀

Start coding or [generate](#) with AI.